Simulating Deutsche Bahn's ICE Network

CS 485: AI and Simulation

Dylan Rybacki

What is DB and it’s ICE network?

- Deutsche Bahn (DB) is a German state-owned railway and logistics company providing both passenger and cargo rail services across Germany, Europe, and the world.
- Founded in 1994
- It’s passenger rail unit is divided into two subdivision:
  - DB Fernverkehr: Provides connections across cities
    - **Intercity Express (ICE)**
    - Intercity (IC)
  - DB Regio: Regional/suburban transit
    - DB Regio Nord, Bayern, etc.
    - S-Bahns

What is DB and it’s ICE network?

- DB Intercity Express (ICE) provides high sped rail travel across longer distances
- Trains can travel up to 200 mph
- Services to many countries outside of Germany including Denmark, Austria, Switzerland, France, etc.
- Major hubs include Munich, Berlin, Frankfurt

Why simulate DB?

- DB was formerly considered one of Europe's most punctual rail networks, yet is now subject to online mockery due to high delay and cancellation rates , especially when compared to competitor networks like Switzerland's SBB or Austria's ÖBB
- Simulating can help understand:
  - Where are the painpoints in the network?
  - How can an unfortunate event propagate through the network?

Simulation Design

- Time-Stepped Agent Based Model
- 1 Simulation Tick = 1 Minute
- Network based approach
  - Nodes represent **stations**
  - Edges represent **tracks/lines** between stations
  - ASSUMPTION: Edges are inferred from timetables and doesn't consider trains passing through stations without stopping
  - Agents will be the **trains**
- Chose to use ABM instead of DES due to more straightforward implementation and logic adjustments

Simulation ICE Network
Typical tuesday

Data Availability

- DB timetables are readily available, and are formatted quite nicely in the data sets provided by https://gtfs.de/en/feeds/de_fv/ which puts the timetables in the General Transit Feed Specification (GTFS) form.
  - Provides lat/lons for stations
  - Gives typical schedules for each day of the week and also holidays
- Station capacity is typically public, but no standardized data could be found
  - ASSUMPTION: Find capacities of major hubs. Non-hubs default of capacity = 2
- Historical delays/cancellations are also available, but will not be used for the simulation to see what causes delays rather than how delays affect the network.

Stations / Nodes

- Nodes store the following information:
  - Name, location, id, capacity
  - Trains currently in the station
  - Tracking metrics like delays caused by the nodes
  - Has a down_time for analysis where we can “shut down” a station
- Logic:
  - If a node is at capacity, it cannot accept another train
  - If a train is accepted, add it to the trains in the station

```python
class Node:
    def __init__(self, name, lat, long, id, capacity = 2):
        self.name = name
        self.lat = lat
        self.long = long
        self.id = id
        self.capacity = capacity

        self.current_trains = []

        self.delay_caused_ticks = 0

        self.down_time = 0

    def atCapacity(self):
        return len(self.current_trains) < self.capacity and self.down_time == 0

    def arrival(self, train):
        self.current_trains.append(train)

    def departure(self, train):
        if(train not in self.current_trains):
            print(f"Warning: Train {train.train_id} not in station {self.name}...")
            pass
        else:
            self.current_trains.remove(train)

    def nodeDown(self):
        if self.down_time > 0:
            self.down_time -= 1
```

Train Tracks / Edges

- Edges store the following info:
  - The nodes it connects
  - Track capacity
  - A FIFO queue for trains waiting to enter the line
  - Trains currently travelling the line
  - ASSUMPTION: Train lines have a default capacity of 15, but this does not a perfect representation of **block signaling**
  - Down time due to maintenance of failures
- Logic:
  - Trains can enter if current trains on edge don't exceed the capacity
  - Tracks can go down for many reasons such as debris or other failures. No trains can enter tracks at this time
  - ASSUMPTION: Every edge has a 0.0001% of going down every tick; down times follow Lognorm(3.4, 0.6) which gives a median repair time of 30mins

```python
class Edge:
    def __init__(self, outboundStation, inboundStation, capacity = 15, failure_rate=0.000001):
        self.name = f'{outboundStation.name} - {inboundStation.name}'
        self.outbound = outboundStation
        self.inbound = inboundStation
        self.capacity = capacity
        self.queue = deque() #FIFO queue for trains waiting to enter line
        self.occupants = [] #List of train(s) on line

        #Stochastic element to model track failure/maitnance needed (parameter to be tuned)
        self.failure_rate = failure_rate
        self.down_time = 0        # Down time to track whether track is operational or how many mins/ticks left until operational

        self.delay_caused_ticks = 0


    def isOpen(self):
        return len(self.occupants) < self.capacity and self.down_time == 0

    def trainToQueue(self, train):
        if train in self.outbound.current_trains:
            self.queue.append(train)

    def acceptTrain(self):
        if len(self.queue) > 0 and self.isOpen():
            activeTrain = self.queue.popleft() #Allow new train to pass through
            self.outbound.departure(activeTrain) #Remove train from station
            self.occupants.append(activeTrain) #Add train to active on track

            # TODO: Implement enter_track on train
            activeTrain.enter_track(self)

    def track_maintenance(self):
        if self.down_time > 0:
            self.down_time -= 1
        else: #once down, no other failure will occur on that track
            if random.random() < self.failure_rate: #tracks have a failure rate of 0.0001% / min

                raw_repair_time = random.lognormvariate(3.4, 0.6) #Maintence time should follow Lognormal with mean of 3.4 and
                                                                  #stdev of 0.6 which gives a median repair time of approx 30 mins
                                                                  # and average of approx. 36 mins
                # Ensure it takes at least 1 tick, and round to a whole integer
                self.down_time = max(1, int(raw_repair_time))

                print(f"[TRACK FAILURE] {self.name} down; estimated repair: {self.down_time} ticks.")
```

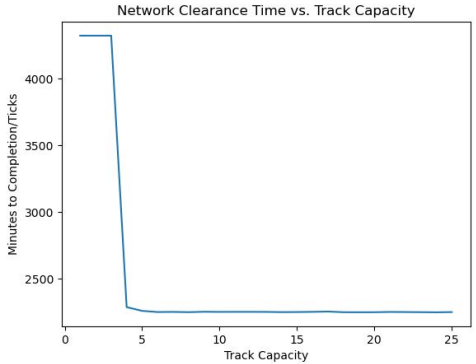
Trains / Agents

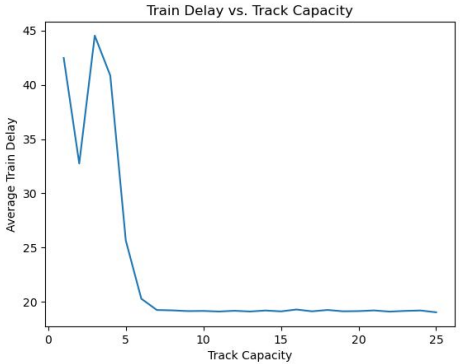
- Trains store the following info:
  - Time table info including how long it takes to travel on edge, how long to dwell, delay statistics, current location, status
- Logic:
  - ASSUMPTION: Trains spawn in 15 mins before schedule initial departure; removed from simulation instantly after completion
  - ASSUMPTION: Train's travel time on an edge is calculated by difference between arrival and departure
  - ASSUMPTION: At each stop, trains have a 5% chance of stalling longer than scheduled due to passenger loading issues or other reasons
  - ASSUMPTION: Trains will never cancel
  - Trains will only leave node once accepted into the station

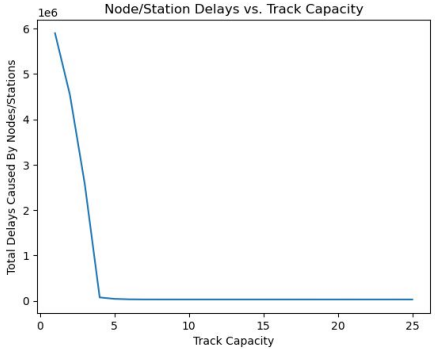
```python
class Train:
    def __init__(self, train_id, scheduled_route, spawn_tick):
        self.train_id = train_id
        self.name = f"ICE {train_id}"

        #FORMAT: [(Edge_Object, travel_ticks, dwell_ticks_at_next_station), ...]
        self.scheduled_route = scheduled_route

        self.spawn_tick = spawn_tick

        self.status = "AT_STATION"
        self.onTime = True
        self.total_delay_ticks = 0
        self.actual_travel_times = []

        self.current_leg_index = 0
        self.current_edge = None

        self.time_on_current_track = 0
        self.current_target_time = 0

        self.time_at_station = 0
        self.current_target_dwell = 15 #15 min spawn assumption
```

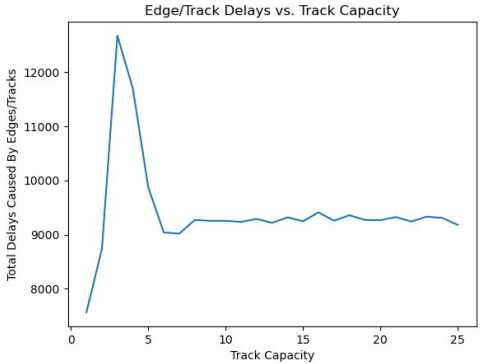
Analysis 1: How does track capacity affect network health?

- Due to simplifications about the edge/track having a consistent capacity throughout the network is inherently flawed as this is often a function of track length and demand.
- This analysis can still show how the capacity between nodes/stations can effect the health of the network.
- Note that the simulation is limited to 72 hours.





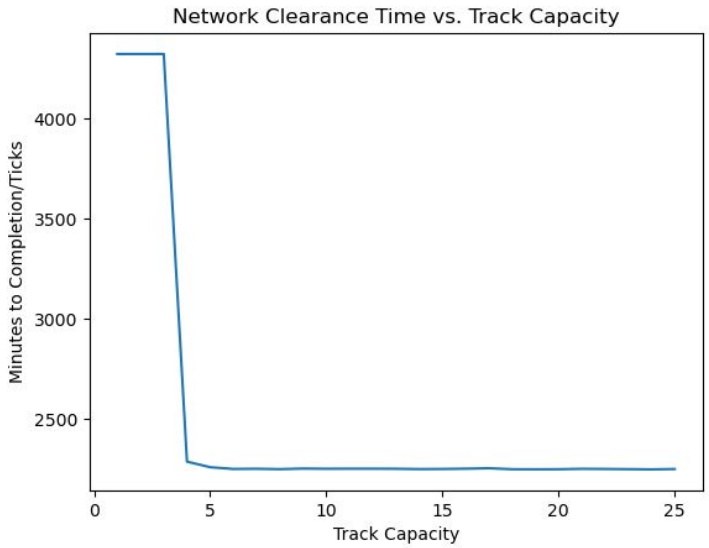




Analysis 1: How does track capacity affect network health?

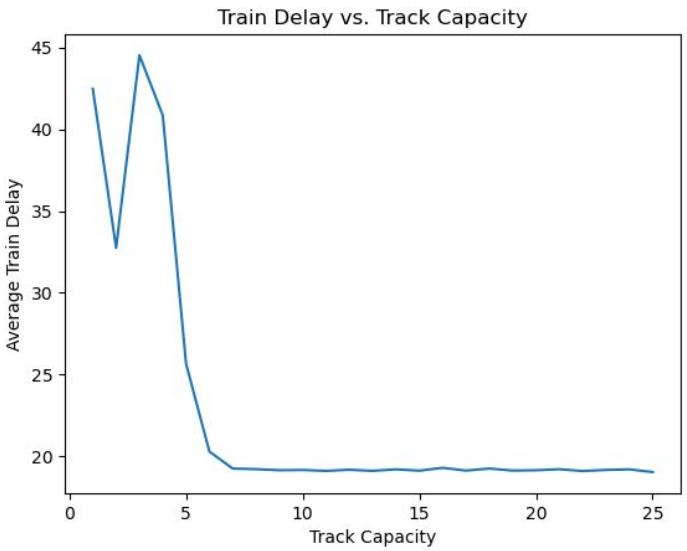
- When track capacities are 3 and under, the simulation appears to get “stuck” where there becomes a bottleneck or a string of bottlenecks where trains aren’t able to finish their lines in the 72 hour simulation period

- Beyond a track capacity of 5, there doesn’t appear to be much of a change when it comes to trains finishing their schedules



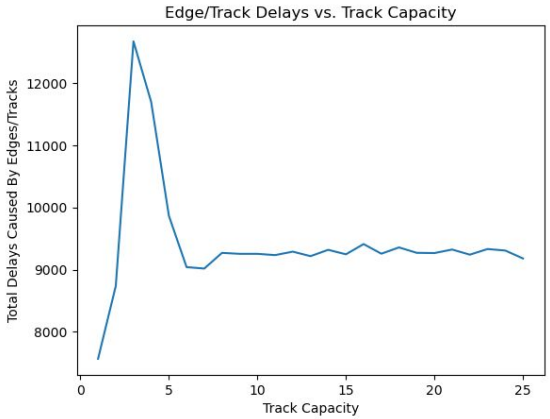
Analysis 1: How does track capacity affect network health?

- This graph shows the average delay over all trains that completed their sequence
  - This is why for track capacities under 4, the average delay looks better, it's really because only the first trains are completing before the network freezes up
- Again shows how after about a track capacity of 7, there are marginal gains to increasing capacity

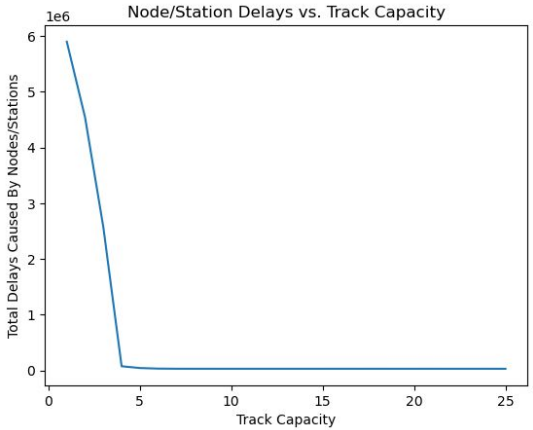


Analysis 1: How does track capacity affect network health?

- Delays caused by edges looks as expected. As edge capacity increases, delays caused by the edge decreases
  - After examining what was causing a consisten delay of 9500 mins, it was mostly comprised of a schedule bug at the Czech/Polish border

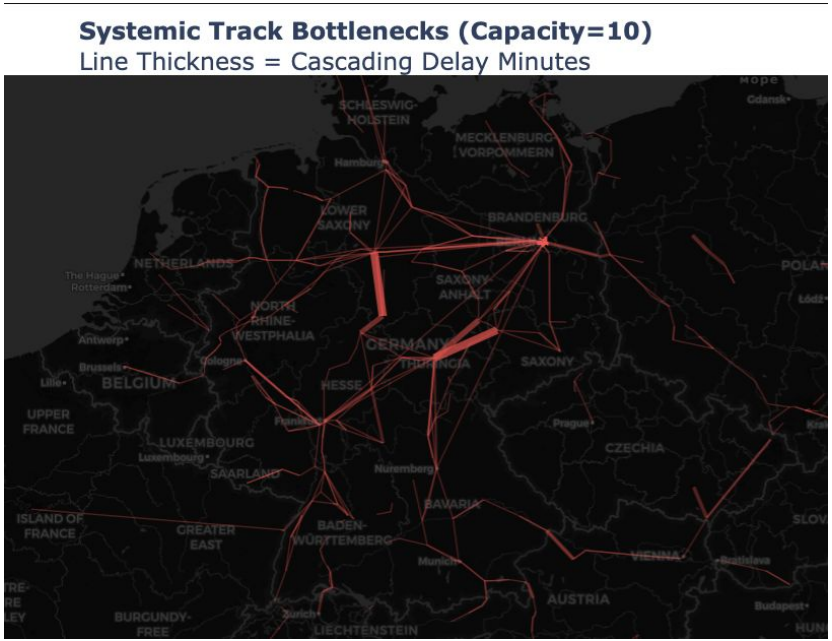


- The simulation showed that there was roughly no delays caused by the stations.
  - This is likely because the we are only simulating ICE (long distance/high-speed rail), but regional services make up a majority of the services at these stations so the station capacity is a non-issue in this simulation



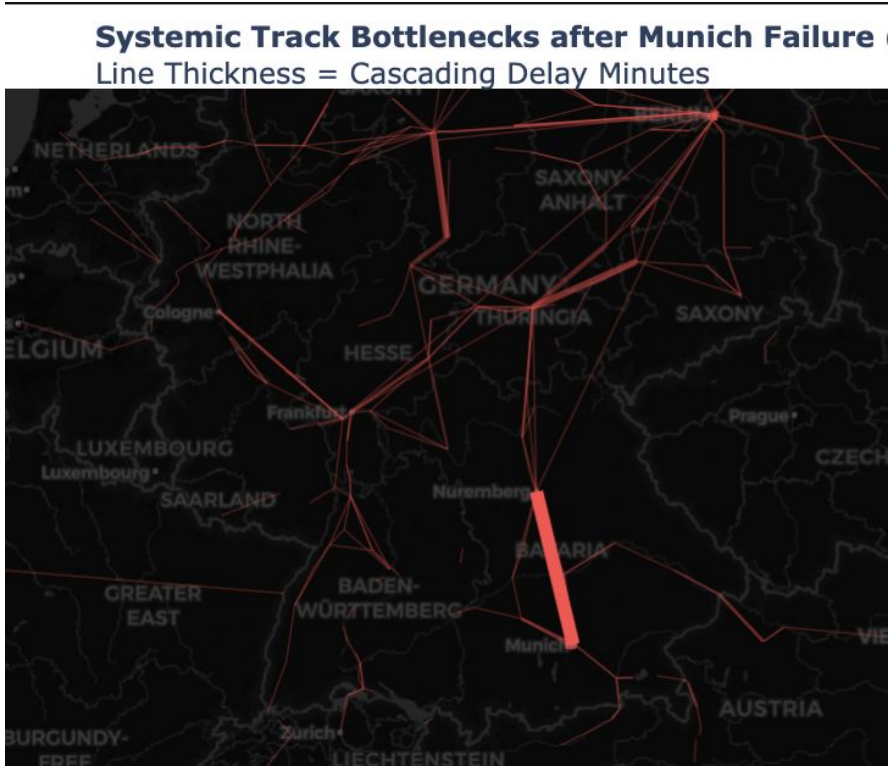
Analysis 2: Where are the bottlenecks occuring?

- Building on the previous analysis, we can visualize what this simulation is having its bottlenecks (assuming edge_capacity = 10)
  - These results are averaged over 100 runs
- Berlin, Halle-Leipzig, and Hannover regions are major culprits for delays
  - Those regions both have very dense network, but a limited number of directions like some place like Frankfurt would so the track bottlenecks get concentrated



Analysis 3: How can a failure propagate through the network?

- Many things can shut down a train station for an extended period of time such as catastrophic events, threats, outages, and more
- To see how an event like this would effect the simulation, I manually shut down München Hbf for 2 hours at 10AM (for 10 runs)
- As expected, the Munich appears to have significantly more delays



Main Takeaways

- Even though this simulation makes a lot of assumptions and greatly oversimplifies the DB network, it is still able to identify some clear flaws and directions worth studying.
- Train networks, especially well established ones like DB, are highly sensitive to both small and large delays
- Future work on this simulation could include adding a cancellation logic to clear up the network when trains get too far behind or adding in the regional network or ghost traffic to see how node congestion effects the network.

Thank You! / Danke!